

BCA 5th Semester

JAVA

Topic : Basics of Java Programming

- By Soudip Sir

2

Basics of Java Programming: Data types, variables, operators, control structures, and loops. Storing of Big Integer. Use of Static variable, final variable, initialization of static and final variable.

Decision Making Statements in Java

Java uses Decision Making Statements to determine the flow of a program based on a condition.

Types:

- if
- if-else
- if-else-if
- Nested if
- Ternary Operator
- Switch

If

```
if(condition)
{
    // true
}
```

If-else

```
if(condition)
{
    // true
}
else
{
    // false
}
```

If-else-if

```
if(condition)
{
    // true
}
else if(condition2)
{
    // true
}
else
{
    // false
}
```

Nested-if

```
if(condition)
{
    if(condition)
    {
    }
    else
    {
    }
}
else
{
    if(condition)
    {
    }
    else
    {
    }
}
```

Switch

```
switch(expression)
{
    case 1:
        break;

    case 2:
        break;

    case 3:
        break;

    default:
}
```

Conditional Statements

Conditional statements are used to execute different blocks of code based on whether a condition is true or false.

Main conditional statements in Java:

- if
- if-else
- if-else-if
- Nested-if
- Switch

If Statement

The if statement is the simplest decision-making statement. It is used to decide whether a particular statement or block of statements will be executed or not.

If the condition is true, the statements inside the if block are executed. If the condition is false, they are not executed.

Syntax

```
if(condition)
{
    // Statements to execute when condition is true
}
```

Execution Flow

Condition → True → Body of if → Next statement

If the condition is false, control goes directly to the next statement.

If-else Statement

The if-else statement is used when we want to execute one block when the condition is true and another block when the condition is false.

Syntax

```
if(condition)
{
    // Executes when condition is true
}
else
{
    // Executes when condition is false
}
```

Execution Flow

- Condition true → Body of if
- Condition false → Body of else

Then the next statement is executed.

If-else-if Ladder

The if-else-if ladder is used when there are multiple conditions/options.

Conditions are checked from top to bottom. When a condition is true, its associated block is executed and the rest of the ladder is bypassed.

If none of the conditions are true, the final else block is executed.

Syntax

```
if(condition1)
{
    // code to be executed if condition1 is true
}
else if(condition2)
{
    // code to be executed if condition2 is true
}
else
{
    // code to be executed if all conditions are false
}
```

One if block can have many else-if blocks, but generally only one else block.

Switch Statement in Java

The switch statement is a multi-way decision statement. It executes different blocks of code according to the value of an expression.

It can be cleaner and more readable compared to many if-else-if conditions.

Syntax

```
switch(expression)
{
    case 1:
        statement;
        break;

    case 2:
        statement;
        break;

    case 3:
        statement;
        break;

    default:
        statement;
}
```

- Using break exits the switch after the matching case is executed.
- default executes when no case matches.

Jump Statements

Java has three main Jump Statements:

- break
- continue
- return

These transfer program control to a different part of the program.

Break

break is mainly used to:

- Terminate the sequence of a switch statement.
- Exit a loop.
- Transfer control flow outside a loop/switch.

Continue

continue skips the remaining part of the current iteration and moves to the next iteration of the loop.

Return

The return statement is used to explicitly return from a method.

It transfers program control back to the caller of the method.

Difference Between Break and Continue

Break	Continue
Terminates the loop immediately.	Skips the current iteration and moves to the next iteration.
Control moves outside the loop.	Control moves to the next loop iteration.
Can be used with both loops and switch.	Can only be used inside loops.
Completely stops further execution of the loop body.	Only skips the remaining statements of the current iteration.
Can be used with labeled loops.	Can also be used with labeled loops.

Loops in Java

A loop is used to execute a set of instructions repeatedly as long as a specified condition remains true.

Java has three main types of loops:

- for loop
- while loop
- do-while loop

Loop in Java

Looping in Java is a feature that helps execute a set of instructions repeatedly.

Loops in Java can be classified in two ways:

1. Entry Controlled Loop

Condition is checked first.

- for
- while

2. Exit Controlled Loop

The loop body executes first, then the condition is checked.

- do-while

Types of Loops

Entry Controlled	Exit Controlled
for	do-while
while	

Basic Syntax

For:

```
for(initialization; condition; update)
{
}
```

While:

```
while(condition)
{
}
```

Do-while:

```
do
{
}
while(condition);
```

For Loop

The for loop is generally used when we know in advance how many times a task needs to be repeated.

The for statement combines together:

- Initialization
- Condition
- Increment/Decrement

Syntax

```
for(initialization; condition; increment/decrement)
{
    // code to be executed
}
```

Execution Flow

- Start
- Initialization
- Condition Checking
- True → Statement
- Increment/Decrement
- Condition Checking (repeats)

When the condition becomes false, the loop ends.

For-each Loop Example

The for-each loop can be used to access each element of an array one by one.

```
class forEach
{
    public static void main(String args[])
    {
        String[] names = {"Sweta", "Gudly", "Amiya"};

        for(String name : names)
        {
            System.out.println("Name: " + name);
        }
    }
}
```

Output:

```
Name: Sweta
Name: Gudly
Name: Amiya
```

While Loop

A while loop is used when the condition needs to be checked before the loop body executes.

Syntax

```
while(condition)
{
    // code to be executed
}
```

Execution Flow

- Start
- Condition Checking
- True → Statement
- Condition Checking (repeats)
- False → End

That is, if the condition is false at the very start, the loop body will not execute even once.

While Loop Example

```
class whileLoop
{
    public static void main(String args[])
    {
        int i = 0;

        while(i <= 10)
        {
            System.out.print(i + " ");
            i++;
        }
    }
}
```

Output:

```
0 1 2 3 4 5 6 7 8 9 10
```

Do-while Loop

The do-while loop ensures that the code block executes at least once, and only then is the condition checked.

Because of this, it is an Exit Controlled Loop.

Syntax

```
do
{
    // code to be executed
}
while(condition);
```

Execution Flow

- Start
- Statement
- Condition Checking
- True → Statement (repeats)
- False → End

Do-while Loop Example

```
class dowhileLoop
{
    public static void main(String[] args)
    {
        int i = 0;

        do
        {
            System.out.print(i + " ");
            i++;
        }
        while(i <= 10);
    }
}
```

Output:

```
0 1 2 3 4 5 6 7 8 9 10
```